

STA 5635 HW 4

Palmer Swanson

February 4, 2026

Problem 1

TISP variable selection method ith threshold operator:

$$\Theta(x, \lambda, \eta) = \begin{cases} 0 & \text{if } |x| \leq \lambda \\ \frac{x}{1+\eta} & |x| > \lambda \end{cases}$$

Penalized logistic regression:

$$L(\boldsymbol{\omega}) = \frac{1}{N} \sum_{j=1}^N \ln(1 + e^{-y_j^* \boldsymbol{\omega}^T \mathbf{x}_j}) + \lambda \sum_{i=1}^p p(\omega_i)$$

assuming $y_j^* \in \{-1, 1\}$ and algorithm iteration:

$$\boldsymbol{\omega}^{t+1} = \Theta \left(\boldsymbol{\omega}^t + \eta' X^T \left[\mathbf{y} - \frac{1}{1 + e^{-X \boldsymbol{\omega}^t}} \right], \lambda \right)$$

assuming $y_j \in \{-1, 1\}$.

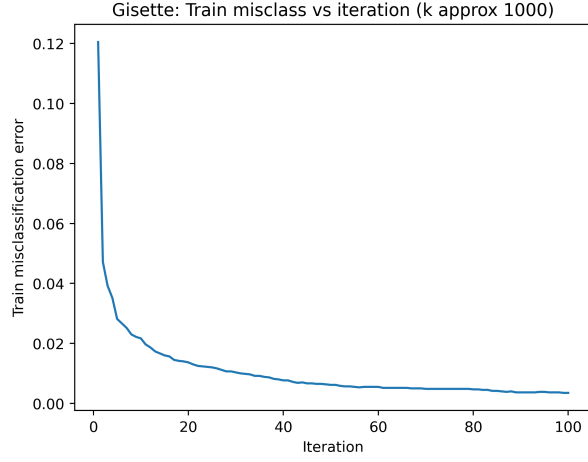
For $\eta = 0$ and 100 iterations, the first portion of question 1 is shown in Figure 1. We search for appropriate λ by performing a grid search across a range of values and selected the value that is closest to the desired number of features.

Second portion of question 1 is shown in Figure 2

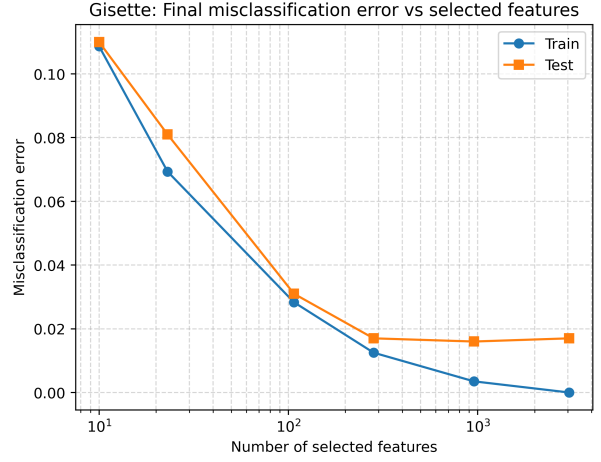
The table reporting the train and test misclassification error, the test AUC, corresponding number of features selected, values of λ , and the training time are reported in Table 1.

Dataset	Target Features	Selected Features	λ	Train Misclass.	Test Misclass.	Test AUC	Train Time (sec)
Gisette	10	10	0.196304	0.108667	0.110000	0.942920	2.046005
Gisette	30	23	0.138489	0.069333	0.081000	0.972464	1.984585
Gisette	100	107	0.086975	0.028333	0.031000	0.993964	1.715003
Gisette	300	282	0.054623	0.012500	0.017000	0.997420	1.760992
Gisette	1000	956	0.019179	0.003500	0.016000	0.998280	2.741985
Gisette	3000	3048	0.004229	0.000000	0.017000	0.997964	1.695035
Dexter	10	12	0.138489	0.116667	0.166667	0.932844	0.382989
Dexter	30	32	0.097701	0.046667	0.116667	0.959911	0.507573
Dexter	100	117	0.068926	0.003333	0.073333	0.975733	0.366997
Dexter	300	263	0.054623	0.000000	0.070000	0.979600	0.380018
Dexter	1000	1210	0.038535	0.000000	0.083333	0.968400	0.595005
Dexter	3000	1739	0.030539	0.000000	0.096667	0.966978	0.376979

Table 1: Train and Test Misclassification Errors, Test AUC, Number of Selected Features, values of λ , and training time, TISP algorithm.

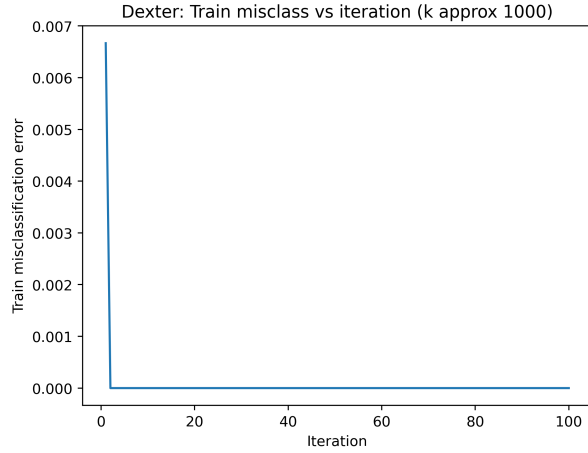


(a) Training misclassification error vs Iteration Number

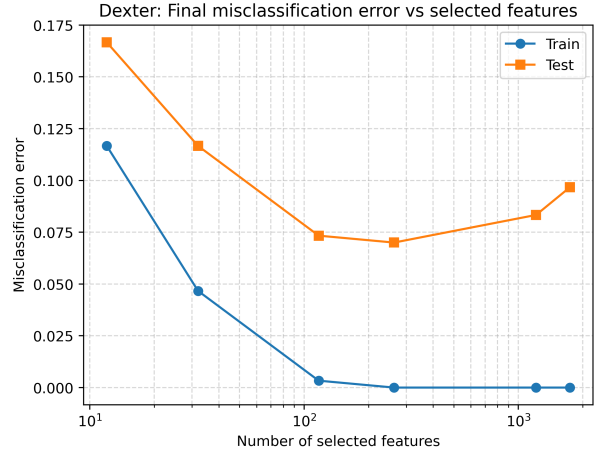


(b) Train and Test Misclassification error vs Number of Selected Features

Figure 1: Train Misclassification Error vs. TISP iteration (left) and Train and Test Misclassification Error vs. Number of Selected Features (right). Gisette Dataset.



(a) Training misclassification error vs Iteration Number



(b) Train and Test Misclassification error vs Number of Selected Features

Figure 2: Train Misclassification Error vs. TISP iteration (left) and Train and Test Misclassification Error vs. Number of Selected Features (right). Dexter Dataset.

Problem 2

FSA variable selection using logistic loss:

$$L_D(\beta) = \frac{1}{N} \sum_{i=1}^N \ln(1 + e^{-y_i \beta^T \mathbf{x}_j}) + s \|\beta\|_2^2$$

We can compute the gradient for one element with a few key facts, and a substitution:
 $u = -y_i \beta^T \mathbf{x}_j$:

$$\begin{aligned} \frac{d}{du} \log(1 + e^u) &= \frac{e^u}{1 + e^u} = \frac{1}{1 + e^{-u}} & \frac{d}{d\beta} u &= -y_i \mathbf{x}_i \\ \frac{d}{d\beta} \log(1 + e^u) &= \frac{1}{1 + e^{-u}} (-y_i \mathbf{x}_i) \end{aligned}$$

substitution gives us:

$$\frac{d}{d\beta} \log(1 + e^{-y_i \beta^T \mathbf{x}_j}) = \frac{1}{1 + e^{y_i \beta^T \mathbf{x}_j}} (-y_i \mathbf{x}_i) + 2s\beta$$

With this, we can get the gradient for all N:

$$\nabla L_D(\beta) = \frac{-1}{N} \sum_{i=1}^N \frac{(y_i \mathbf{x}_i)}{1 + e^{y_i \beta^T \mathbf{x}_j}}$$

We also use an inverse scheduler with parameter μ :

$$M_i = k + (p - k) \max \left(0, \frac{N^{iter} - 2i}{2i\mu + N^{iter}} \right)$$

For $s = 0.001$, $\mu = 100$, $\eta = 0.1$ and 100 iterations, the first portion of question 2 is shown in Figure 3:

The table reporting train and test misclassification errors, test AUC, and training time is shown in Table 2:

To compare the computation time of the FSA to TISP, I computed the ratio of FSA computation time to TISP. For the 30 and 1000 features cases, FSA was faster. Also, for the 10 feature Gisette case, FSA was faster. For all others, TISP was faster. I did not include this table to abide by submission guidelines.

We can also include the train/test misclassification error rate vs. k for the two datasets in Figure 4:

The two methods appear to be quit similar, with only slight deviations in trajectories.

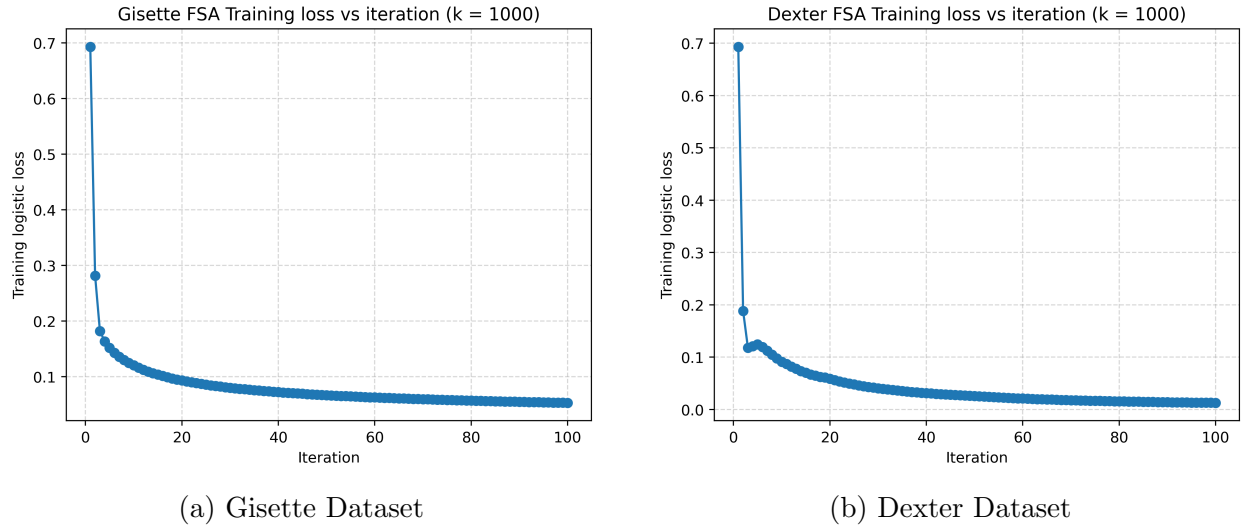
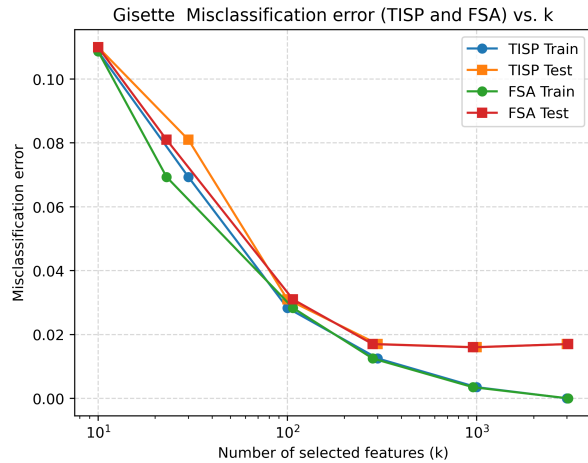


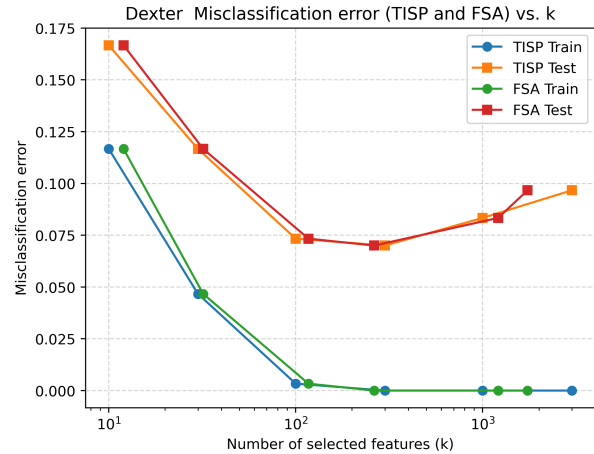
Figure 3: Plot of Training Loss vs. Iteration Number for $k = 1000$. Gisette Dataset (left), Dexter Dataset (right).

Dataset	Target k	Selected	Train Misclass.	Test Misclass.	Test AUC	Time (s)
Gisette	10	10	0.122833	0.131000	0.937842	1.748005
Gisette	30	30	0.061667	0.077000	0.981482	1.795004
Gisette	100	100	0.030333	0.030000	0.994576	1.832055
Gisette	300	300	0.017000	0.017000	0.997848	1.877003
Gisette	1000	1000	0.011667	0.017000	0.998140	1.849008
Gisette	3000	3000	0.003333	0.018000	0.997900	1.801003
Dexter	10	10	0.110000	0.153333	0.945711	0.488000
Dexter	30	30	0.040000	0.100000	0.960711	0.482002
Dexter	100	100	0.006667	0.093333	0.975067	0.476003
Dexter	300	300	0.000000	0.090000	0.971511	0.472001
Dexter	1000	1000	0.000000	0.090000	0.965289	0.475001
Dexter	3000	3000	0.000000	0.096667	0.946578	0.472998

Table 2: Train and Test Misclassification Errors, Test AUC, and training time, FSA algorithm.



(a) Gisette Dataset



(b) Dexter Dataset

Figure 4: Plot of Training Loss vs. Target Features. Gisette Dataset (left), Dexter Dataset (right).

Code

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4 import time
5 import matplotlib.pyplot as plt
6 from sklearn.metrics import roc_auc_score, roc_curve
7
8 #Q1 datasets
9 q1_datasets = [
10     {
11         "name": "Gisette",
12         "X_train": "assignments/hw04/data/gisette_train.data",
13         "y_train": "assignments/hw04/data/gisette_train.labels",
14         "X_valid": "assignments/hw04/data/gisette_valid.data",
15         "y_valid": "assignments/hw04/data/gisette_valid.labels",
16         "missclass_1000_out":
17             "assignments/hw04/figures/gisette_misclass_1000.png",
18         "FSA_out": "assignments/hw04/figures/gisette_misclass_1000_FSA.png"
19     },
20     {
21         "name": "Dexter",
22         "X_train": "assignments/hw04/data/dexter_train.csv",
23         "y_train": "assignments/hw04/data/dexter_train.labels",
24         "X_valid": "assignments/hw04/data/dexter_valid.csv",
25         "y_valid": "assignments/hw04/data/dexter_valid.labels",

```

```

25     "missclass_1000_out":
26     "assignments/hw04/figures/dexter_misclass_1000.png",
27     "FSA_out": "assignments/hw04/figures/dexter_misclass_1000_FSA.png"
28 }
29 ]
30 #thresholding function, w is vector of length p+1
31 #I don't think we should penalize the intercept
32 #https://stats.stackexchange.com/questions/559416/ridge-regression-subtlety-on-intercept
33 #will element wise decide whether or not to select variable
34 def threshold_operator(ws, lamb):
35     #make copy of w vector
36     w_cop = ws.copy()
37
38     #https://numpy.org/devdocs/reference/generated/numpy.ones_like.html
39     #plus don't do anything to the intercept
40     #We should just get a boolean vector (F, T, T, T, T, etc)
41     mask = np.ones_like(w_cop, dtype=bool)
42     mask[0] = False
43
44     #threshold booleanwise
45     w_cop[mask & (np.abs(w_cop) <= lamb)] = 0.0
46
47     return w_cop
48
49 #Q1 parameters
50     iters = 100
51     eta = 0.0
52     features = [10, 30, 100, 300, 1000, 3000]
53
54     #we're going to be using logspace anyways
55     lambdas_to_search = np.logspace(-5, 0, 100)
56
57     results = []
58     #Q1
59     for d in q1_datasets:
60         #get name of dataset
61         name = d["name"]
62         print(name)
63
64         #Load datasets. use default read.csv if Dexter dataset
65         if name == "Dexter":
66             X_train = pd.read_csv(d["X_train"], header=None)
67             X_valid = pd.read_csv(d["X_valid"], header=None)
68         else:
69             X_train = pd.read_csv(d["X_train"], delim_whitespace=True, header=None)
70             X_valid = pd.read_csv(d["X_valid"], delim_whitespace=True, header=None)

```

```

71 y_train = pd.read_csv(d["y_train"], header=None).values.ravel()
72 y_valid = pd.read_csv(d["y_valid"], header=None).values.ravel()
73
74
75 #convert labels to 0/1 for later
76 y_train_converted_01 = (y_train == 1).astype(int)
77 y_valid_converted_01 = (y_valid == 1).astype(int)
78
79 #standardize
80 scaler = StandardScaler()
81 X_train_scaled = scaler.fit_transform(X_train)
82 X_test_scaled = scaler.transform(X_valid)
83
84 #add column of 1s to augment w
85 X_train_tilde = np.column_stack([np.ones(len(X_train_scaled)),
86 X_train_scaled])
87 X_valid_tilde = np.column_stack([np.ones(len(X_test_scaled)), X_test_scaled])
88
89 #get N, p
90 N, p = X_train_tilde.shape
91
92 #from notes, eta' can be 1/N
93 eta_prime = 1.0/N
94
95 #ok so what are we doing here. We can to find a lambda that gets us the
96 closest to some number
97 #of features. first thing we need to do is loop over the number of possible
98 features.
99 #going to be at least 2 loops
100 #basically a grid search over the possible lambda values
101 for k in features:
102     print(k)
103     best_lambda = None
104     best_delta = None
105     best_selected_vars = None
106
107     #this loops just searches for the best lambda
108     for j in lambdas_to_search:
109         #print(j)
110
111         #initialize w to 0s
112         w = np.zeros(p)
113
114         #loop over the number of iterations
115         for i in range(iters):
116             #get predictor
117             z = X_train_tilde @ w

```

```

115         #build fractional part
116         frac = 1.0/(1.0 + np.exp(-z))
117
118         #build y- fractional part
119         difference = y_train_converted_01 - frac
120
121         #get the update portion
122         update_port = X_train_tilde.T @ difference
123
124         #temporary new omega and do the thresholding
125         temp = w + (eta_prime*update_port)
126         w = threshold_operator(temp, j)
127
128         #need to count how many variables were selected
129         #don't count intercept and give a small tolerance
130         #check to see if we are close to the desired number of features
131         k_selected = int(np.sum(np.abs(w[1:]) > 1e-10))
132         delta = abs(k_selected - k)
133
134         #if we don't have a best distance, or if the gap is smaller than the
135         previous
136         #best, replace gap, lambda, number of variables selected
137         if (best_delta is None) or (delta < best_delta):
138             best_delta = delta
139             best_lambda = j
140             best_selected_vars = k_selected
141
142         #ok so now we have the lambdas. Use them to run this again
143         #now loop over everything again to time plot and get misclass rates
144         w = np.zeros(p)
145
146         #need to store misclass error only when trying to select 1000 features
147         misclass_1000 = []
148
149         start = time.time()
150         for t in range(iters):
151             z = X_train_tilde @ w
152             frac = 1.0 / (1.0 + np.exp(-z))
153             difference = (y_train_converted_01 - frac)
154             update_port = X_train_tilde.T @ difference
155             temp = w + (eta_prime*update_port)
156             w = threshold_operator(temp, best_lambda)
157
158             #need to track within this iterations loop
159             #now how to do that
160             if k == 1000:

```



```

161         z_new = X_train_tilde @ w
162         p_train_1 = np.exp(-np.logaddexp(0.0, -z_new))
163         p_train_0 = np.exp(-np.logaddexp(0.0, z_new))
164         odds_train = p_train_1/p_train_0
165         yhat_train = (odds_train > 1.0).astype(int)
166         train_err = np.mean(yhat_train != y_train_converted_01)
167
168         #append to misclass_1000
169         misclass_1000.append(train_err)
170
171     train_time = time.time() - start
172
173     k_selected = int(np.sum(np.abs(w[1:]) > 1e-10))
174
175     #train and test misclass errors
176     z_train = X_train_tilde @ w
177     z_test = X_valid_tilde @ w
178
179     #get probabilities for 1 and 0
180     p_train_1 = np.exp(-np.logaddexp(0.0, -z_train))
181     p_train_0 = np.exp(-np.logaddexp(0.0, z_train))
182
183     p_test_1 = np.exp(-np.logaddexp(0.0, -z_test))
184     p_test_0 = np.exp(-np.logaddexp(0.0, z_test))
185
186     #compute the odds of train and test
187     odds_train = p_train_1/p_train_0
188     odds_test = p_test_1/p_test_0
189
190     #make predictions. Predcit 1 if Odds > 1
191     #as.int makes things nicer
192     yhat_train = (odds_train > 1.0).astype(int)
193     yhat_test = (odds_test > 1.0).astype(int)
194
195     #misclassification error
196     train_err = np.mean(yhat_train != y_train_converted_01)
197     test_err = np.mean(yhat_test != y_valid_converted_01)
198
199     #compute AUC
200     test_auc = roc_auc_score(y_valid_converted_01, X_valid_tilde@w)
201
202     #plotting the misclass_error vs iterations for 1000 features
203     if k == 1000:
204         plt.figure()
205         plt.plot(np.arange(1, iters + 1), misclass_1000)
206         plt.xlabel("Iteration")
207         plt.ylabel("Train_misclassification_error")

```

```

208         plt.title(f"{name}: Train_misclass_vs_iteration_{k_approx_1000}")
209         plt.savefig(d["missclass_1000_out"], dpi=400, bbox_inches="tight")
210         plt.close()
211         #plt.show()
212
213     #append to results
214     results.append({
215         "dataset": name,
216         "target_features": k,
217         "selected_features": k_selected,
218         "selected_lambda": best_lambda,
219         "train_misclass": train_err,
220         "test_misclass": test_err,
221         "test_auc": test_auc,
222         "train_time_sec": train_time
223     })
224
225 #df for both Gisette and Dexter
226 df_q1 = pd.DataFrame(results)
227
228 #rename columns. LATEX wasn't liking the column names
229 df_q1_latex = df_q1.rename(columns={
230     "dataset": "Dataset",
231     "target_features": "Target_Features",
232     "selected_features": "Selected_Features",
233     "selected_lambda": r"$\lambda$",
234     "train_misclass": "Train_Misclass.",
235     "test_misclass": "Test_Misclass.",
236     "test_auc": "Test_AUC",
237     "train_time_sec": "Train_Time_(sec)"
238 })
239
240 #need to send this table to LATEX
241 LATEX_table = df_q1_latex.to_latex(
242     index=False,
243     caption=None,
244     label=None,
245     column_format="c" * df_q1_latex.shape[1],
246     escape=False
247 )
248
249 #save latex table
250 with open("assignments/hw04/output/misclass_table.tex", "w") as f:
251     f.write(LATEX_table)
252
253 #plot the final train and test misclassification error vs # of selected features
254 #using a semilog axis

```

```

255 for name in df_q1["dataset"].unique():
256     #create filename to save plot
257     outpath = f"assignments/hw04/figures/{name}_misclass_vs_features.png"
258
259     #select the results for a given dataset
260     sub = df_q1[df_q1["dataset"] == name].copy()
261
262     #works better sorted
263     #sub = sub.sort_values("selected_features")
264
265     plt.figure()
266     plt.semilogx(sub["selected_features"].to_numpy(),
267                 sub["train_misclass"].to_numpy(), marker="o", label="Train")
268     plt.semilogx(sub["selected_features"].to_numpy(),
269                 sub["test_misclass"].to_numpy(), marker="s", label="Test")
270
271     plt.xlabel("Number_of_selected_features")
272     plt.ylabel("Misclassification_error")
273     plt.title(f"{name}: Final misclassification_error_vs_selected_features")
274     plt.legend()
275     plt.grid(True, which="both", linestyle="--", alpha=0.5)
276     plt.savefig(outpath, dpi=400, bbox_inches="tight")
277     plt.close()
278     #plt.show()
279
280 #Q2, using same datasets
281 q2_datasets = q1_datasets
282
283 s = 0.001
284 mu = 100
285 iters = 100
286 features = [10, 30, 100, 300, 1000, 3000]
287 #eta=20 was too big. Overflow error
288 eta = 0.1
289
290 results = []
291
292 for d in q2_datasets:
293     #get name of dataset
294     name = d["name"]
295     print(name)
296
297     #Load datasets. use default read.csv if Dexter dataset
298     if name == "Dexter":
299         X_train = pd.read_csv(d["X_train"], header=None)
300         X_valid = pd.read_csv(d["X_valid"], header=None)
301     else:

```

```

300     X_train = pd.read_csv(d["X_train"], delim_whitespace=True, header=None)
301     X_valid = pd.read_csv(d["X_valid"], delim_whitespace=True, header=None)
302
303     #need to check that y is +1 or -1. If loop?
304     y_train = pd.read_csv(d["y_train"], header=None).values.ravel()
305     y_valid = pd.read_csv(d["y_valid"], header=None).values.ravel()
306
307     #need to force these ya values to be -1/+1
308     #https://numpy.org/devdocs/reference/generated/numpy.where.html
309     y_train = np.where(y_train > 0, 1.0, -1.0)
310     y_valid = np.where(y_valid > 0, 1.0, -1.0)
311
312     #convert labels to 0/1 for likelihood to work with AUC
313     y_train_converted_01 = (y_train == 1).astype(int)
314     y_valid_converted_01 = (y_valid == 1).astype(int)
315
316     #standardize, not forgetting this part this time
317     scaler = StandardScaler()
318     X_train_scaled = scaler.fit_transform(X_train)
319     X_test_scaled = scaler.transform(X_valid)
320
321     #add column of 1s to augment w
322     X_train_tilde = np.column_stack([np.ones(len(X_train_scaled)),
323     X_train_scaled])
324
325     X_valid_tilde = np.column_stack([np.ones(len(X_test_scaled)), X_test_scaled])
326
327     #get N, p
328     N, p = X_train_tilde.shape
329
330     training_loss_1000 = []
331
332     #loop over the number of possible features
333     for feat in features:
334         #initialize to 0
335         beta = np.zeros(p)
336
337         start = time.time()
338
339         for i in range(1000):
340             #get predictor
341             z = X_train_tilde @ beta
342             yz = y_train * z
343
344             #need numpy function I found
345             #https://numpy.org/devdocs/reference/generated/numpy.log1p.html
346             #adding penalty component
347             exponent_part = np.exp(-yz)

```

```

346     log_part = np.log1p(exponent_part)
347     penalty = s*np.sum(beta[1:]**2)
348     loss = np.mean(log_part) + penalty
349
350     #keep track of the 1000 feature case
351     if feat == 1000:
352         training_loss_1000.append(loss)
353
354     #no sparsity penalty
355     #compute gradient. Per quiz and notes, FSA does NOT use a sparsity
penalty for feature
356     #selection
357     part1 = 1.0 / (1.0 + np.exp(yz))
358     grad = -(1/N)*(X_train_tilde.T @ (y_train*part1))
359
360     #penalty component, not penalizing intercept
361     pen_grad = 2*s*beta
362     pen_grad[0] = 0.0
363
364     grad = grad + pen_grad
365
366     #update Beta using gradient step
367     beta = beta - (eta*grad)
368
369     #inverse scheduler
370     denom = (2.0*i*mu) + iters
371     numer = (iters - (2*i))
372     ratio = numer/denom
373
374     #get maximum of 0 or the ratio. Exclude the intercept
375     p_feat = p-1
376     max_portion = max(0, ratio)
377     Mi = np.round(feat + (p_feat-feat)*max_portion)
378     Mi = int(Mi)
379
380     #need to keep the top Mi variables, not including intercept
381     #sort. Keep the largest Mi.
382     #beta length p. beta[0] = intercept. beta[1:] feature coefficients
383     #Mi = # of features permitted at this iteration
384     #https://numpy.org/doc/stable/reference/generated/numpy.argsort.html
385     b = beta[1:]
386     abs_b = abs(b)
387
388     #first entries in sort will be the smallest. Need to set all but
the last
389     #Mi to 0 aka we need to set the first p_feat-Mi to 0
390     sort = np.argsort(abs_b)

```

```

391         b[sort[: -Mi]] = 0.0
392         beta[1:] = b
393         #should end up with something that looks like
394         #beta = [1.0, 0.0, 3.0, 5.0, -3.0, 0.0, 0.0, 0.0,...]
395
396     end = time.time()
397     run_time = end-start
398
399     #gotta be a better way to do this stuff
400     #need to make predictions now for misclass and AUC
401     z_train = X_train_tilde @ beta
402     z_test = X_valid_tilde @ beta
403
404     #get probabilities for 1 and 0
405     p_train_1 = np.exp(-np.logaddexp(0.0, -z_train))
406     p_train_0 = np.exp(-np.logaddexp(0.0, z_train))
407
408     p_test_1 = np.exp(-np.logaddexp(0.0, -z_test))
409     p_test_0 = np.exp(-np.logaddexp(0.0, z_test))
410
411     #compute the odds of train and test
412     odds_train = p_train_1/p_train_0
413     odds_test = p_test_1/p_test_0
414
415     #make predictions. Predcit 1 if Odds > 1
416     #as.int makes things nicer
417     yhat_train = (odds_train > 1.0).astype(int)
418     yhat_test = (odds_test > 1.0).astype(int)
419
420     #misclassification error
421     train_err = np.mean(yhat_train != y_train_converted_01)
422     test_err = np.mean(yhat_test != y_valid_converted_01)
423
424     #compute AUC
425     test_auc = roc_auc_score(y_valid_converted_01, X_valid_tilde@beta)
426
427     #compute how many variables are non 0 aka selected
428     selected = int(np.sum(beta[1:] != 0))
429     print(selected)
430
431     #append to results
432     results.append({
433         "dataset": name,
434         "target_features": feat,
435         "selected_features": selected,
436         "train_misclass": train_err,
437         "test_misclass": test_err,

```

```

438         "test_auc": test_auc,
439         "train_time_sec": run_time
440     })
441
442     #plot the training loss vs iteration number
443     plt.figure()
444     plt.plot(range(1, len(training_loss_1000) + 1), training_loss_1000,
445              marker="o")
446     plt.xlabel("Iteration")
447     plt.ylabel("Training_logistic_loss")
448     plt.title(f"{name}_FSA_Training_loss_vs_iteration_(k={1000})")
449     plt.grid(True, linestyle="--", alpha=0.5)
450     plt.savefig(d["FSA_out"], dpi=400, bbox_inches="tight")
451     plt.close()
452     #plt.show()
453
454     #df for both Gisette and Dexter
455     df_q2 = pd.DataFrame(results)
456
457     #renaming this table for LATEX
458     df_q2_latex = df_q2.rename(columns={
459         "dataset": "Dataset",
460         "target_features": "Target_$$$k$",
461         "selected_features": "Selected",
462         "train_misclass": "Train_Misclass.",
463         "test_misclass": "Test_Misclass.",
464         "test_auc": "Test_AUC",
465         "train_time_sec": "Time_(s)"
466     })
467
468     #need to send this table to LATEX
469     LATEX_table = df_q2_latex.to_latex(
470         index=False,
471         caption=None,
472         label=None,
473         column_format="c" * df_q2_latex.shape[1],
474         escape=False
475     )
476
477     #save latex table
478     with open("assignments/hw04/output/misclass_table_q2.tex", "w") as f:
479         f.write(LATEX_table)
480
481     #need to plot the train and test errors vs k
482     for name in df_q2["dataset"].unique():
483         outpath = f"assignments/hw04/figures/{name}_misclass_TISP_vs_FSA_.png"
484         #make copy of the df

```

```

484     sub1 = df_q1[df_q1["dataset"] == name].copy()
485     sub2 = df_q2[df_q2["dataset"] == name].copy()
486
487     plt.figure()
488     #TISP
489     plt.semilogx(sub2["selected_features"].to_numpy(),
490                 sub1["train_misclass"].to_numpy(), marker="o", label="TISP_Train")
491     plt.semilogx(sub2["selected_features"].to_numpy(),
492                 sub1["test_misclass"].to_numpy(), marker="s", label="TISP_Test")
493     #FSA
494     plt.semilogx(sub1["selected_features"].to_numpy(),
495                 sub1["train_misclass"].to_numpy(), marker="o", label="FSA_Train")
496     plt.semilogx(sub1["selected_features"].to_numpy(),
497                 sub1["test_misclass"].to_numpy(), marker="s", label="FSA_Test")
498
499     plt.xlabel("Number_of_selected_features(k)")
500     plt.ylabel("Misclassification_error")
501     plt.title(f"{name}_Misclassification_error_(TISP_and_FSA)_vs._k")
502     plt.legend()
503     plt.grid(True, linestyle="--", alpha=0.5)
504     plt.savefig(outpath, dpi=400, bbox_inches="tight")
505     plt.close()
506
507     #want to compare the computation time for the two methods.
508     #not included in the writeup, but I did it.
509     df_q1["method"] = "TISP"
510     df_q2["method"] = "FSA"
511     df_time = pd.concat([df_q1, df_q2], ignore_index=True)
512
513     #need to pivot. Chaining this stuff together like the pipe operator in R
514     df_time = (
515         df_time[["dataset", "target_features", "method", "train_time_sec"]]
516         .pivot(
517             index=["dataset", "target_features"],
518             columns="method",
519             values="train_time_sec"
520         )
521         .assign(
522             FSA_over_TISP=lambda x: (x["FSA"] / x["TISP"]).round(2)
523         )
524         .reset_index()
525     )
526
527     #rename columns
528     df_time_latex = df_time.rename(columns={
529         "dataset": "Dataset",
530         "target_features": "Target_$k$",

```



```

527     "TISP": "TISP_Time(s)",
528     "FSA": "FSA_Time(s)",
529     "FSA_over_TISP": "FSA/Time"
530 })
531
532 print(df_time_latex)
533
534 latex_table = df_time_latex.to_latex(
535     index=False,
536     escape=False,
537     float_format="%.2f",
538     column_format="c" * df_time_latex.shape[1]
539 )
540
541 with open("assignments/hw04/output/time_comparison.tex", "w") as f:
542     f.write(latex_table)

```